

PRACTICAL_WEEK-12

Task 1: Implement the Producer-Consumer problem using counting semaphores and POSIX threads. Evaluate synchronization correctness and system throughput under different buffer sizes.

```
pthread_mutex_init(&mutex, NULL);

printf("Producer-Consumer Simulation\n");
printf("Buffer Size: %d\n\n", buffer_size);

clock_gettime(CLOCK_MONOTONIC, &start);

pthread_create(&producer_thread,
              NULL,
              producer,
              NULL);

pthread_create(&consumer_thread,
              NULL,
              consumer,
              NULL);

pthread_join(producer_thread, NULL);
pthread_join(consumer_thread, NULL);

clock_gettime(CLOCK_MONOTONIC, &end);

double elapsed =
    (end.tv_sec - start.tv_sec) +
    (end.tv_nsec - start.tv_nsec) / 1000000000.0;

printf("\nAll %d items produced and consumed correctly.\n",
      TOTAL_ITEMS);

printf("Execution Time: %.6f seconds\n", elapsed);

printf("Throughput: %.2f items/second\n",
      TOTAL_ITEMS / elapsed);

pthread_mutex_destroy(&mutex);

sem_close(empty_slots);
sem_close(full_slots);

sem_unlink(empty_name);
sem_unlink(full_name);

free(buffer);

return 0;
}
```

OUTPUT:

```
amrutha@Amruthas-MacBook-Air OSSP % ./bin/practical12_task1 10
Producer-Consumer Simulation
Buffer Size: 10

Produced: 1 at position 0
Produced: 2 at position 1
Produced: 3 at position 2
Produced: 4 at position 3
Produced: 5 at position 4
Produced: 6 at position 5
Produced: 7 at position 6
Produced: 8 at position 7
Produced: 9 at position 8
Produced: 10 at position 9
```

```
All 20 items produced and consumed correctly.
Execution Time: 0.000250 seconds
Throughput: 80000.00 items/second
amrutha@Amruthas-MacBook-Air OSSP %
```

Task 2: Design a deadlock scenario involving multiple threads competing for shared resources. Demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and implement a deadlock prevention strategy using resource ordering.

```
printf("=== Deadlock Demonstration ===\n");

pthread_create(&t1, NULL, deadlock_thread1, NULL);
pthread_create(&t2, NULL, deadlock_thread2, NULL);

pthread_join(t1, NULL);
pthread_join(t2, NULL);
}
else if (strcmp(argv[1], "safe") == 0)
{
    int id1 = 1;
    int id2 = 2;

    printf("=== Deadlock Prevention Using Resource Ordering ===\n");

    pthread_create(&t1, NULL, safe_thread, &id1);
    pthread_create(&t2, NULL, safe_thread, &id2);

    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    printf("\nBoth threads completed successfully.\n");
    printf("Deadlock prevented using consistent resource ordering.\n");
}
else
{
    printf("Invalid option. Use deadlock or safe.\n");
}

pthread_mutex_destroy(&resource1);
pthread_mutex_destroy(&resource2);

return 0;
}
```

OUTPUT:

```
[amrutha@Amruthas-MacBook-Air OSSP % gcc src/practical12_task2.c -o bin/practical12_task2 -pthread]
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/practical12_task2 deadlock]
=== Deadlock Demonstration ===
Thread 1 locking Resource 1...
Thread 2 locking Resource 2...
Thread 1 waiting for Resource 2...
Thread 2 waiting for Resource 1...
```

```
[amrutha@Amruthas-MacBook-Air OSSP % ./bin/practical12_task2 safe]
=== Deadlock Prevention Using Resource Ordering ===
Thread 1 locking Resource 1...
Thread 2 locking Resource 1...
Thread 1 locking Resource 2...
Thread 1 acquired both resources.
Thread 1 released resources.
Thread 2 locking Resource 2...
Thread 2 acquired both resources.
Thread 2 released resources.

Both threads completed successfully.
Deadlock prevented using consistent resource ordering.
```